Reviewer Pack — Minimal Tropical Rank Correlation with SAT Clause Set Comple…

Entry 2dc21e788874 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-31 20:49:51 UTC

Minimal Tropical Rank Correlation with SAT Clause Set Complexity

Entry ID: `2dc21e788874` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-31 20:49:51 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): SAT Clause Sets

Statement:

> The minimal tropical rank (mtr) of a SAT clause set is linearly correlated with its complexity, such that mtr(C) = Θ(|C|^α), where |C| is the number of clauses in the clause set and α is a constant.

Rationale (proposer's reasoning):

> Tropical geometry provides a natural framework for studying the structure of Boolean functions. The minimal tropical rank measures the complexity of these functions in the tropical setting, which may reveal underlying structures related to the complexity of SAT clause sets.

Taxonomy category: `TROPICAL_GEOMETRY` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `bca06ee9011eae2e`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> Minimal tropical rank (mtr) is linearly correlated with clause set complexity if the Pearson correlation coefficient is ≥ 0.7 and α is within ±5% of the computed value.

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
ALGEBRIZATION	SAFE	1.00	SAFE	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_clause_set(n):
        return [random.choice([f"x{i}", f"~x{i}"]) for _ in range(n)]

    def minimal_tropical_rank(clause_set):
        literals = set()
        for clause in clause_set:
            literals.update(clause.split(' '))
        rank = 0
        for literal in literals:
            if literal.startswith('~'):
                rank += 1
        return rank

    def pearson_correlation(ranks, n_values):
        mean_ranks = sum(ranks) / len(ranks)
        mean_n = sum(n_values) / len(n_values)
        numerator = sum((r - mean_ranks) * (n - mean_n) for r, n in zip(ranks, n_values))
        denominator = math.sqrt(sum((r - mean_ranks)**2 for r in ranks)) * math.sqrt(sum((n - mean
        return numerator / denominator if denominator != 0 else 0

    ranks = []
    n_values = []

    for n in [5, 10, 15, 20, 30, 40]:
        clause_sets = [generate_clause_set(n) for _ in range(30)]
        for clause_set in clause_sets:
            mtr = minimal_tropical_rank(clause_set)
            ranks.append(mtr)
            n_values.append(n)

    correlation_coefficient = pearson_correlation(ranks, n_values)
    alpha = math.log(correlation_coefficient) / math.log(len(ranks))
```

```
    metric_name = "Pearson Correlation Coefficient"
    metric_value = correlation_coefficient
    instances_tested = len(ranks)
    n_max = max(n_values)
    conjecture_holds = abs(correlation_coefficient) >= 0.7 and -0.05 <= alpha - correlation_coeffi
    counterexample = "" if conjecture_holds else f"alpha={alpha:.2f} (expected α within ±5%)"

    return {
        "metric_name": metric_name,
        "metric_value": metric_value,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value)**2 for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value:.4f} std={std_value:.4f} support_fraction={supp
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"alpha out of bounds\" first_failing_seed={first
    else:
        print("RESULT: INCONCLUSIVE reason=unreachable")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46ee61fb.py", line 76, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46ee61fb.py", line 45, in run_trial
    clause_sets = [generate_clause_set(n) for _ in range(30)]
                   ^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_46ee61fb.py", line 22, in generate_clau
    return [random.choice([f"x{i}", f"~x{i}"]) for _ in range(n)]
```

```
                                ^
NameError: name 'i' is not defined. Did you mean: 'id'?
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test code crashed due to a NameError, which prevented it from producing any data or results. | next: Debug the test code to fix the NameError and rerun the test to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17260
2	preregistration	ollama_remote	glm4:latest	0	0	34415
3	novelty	ollama_remote	glm4:latest	0	0	15278
4	novelty	ollama_remote	glm4:latest	0	0	8994
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19205
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9493
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17959
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9755
9	judge	ollama_remote	glm4:latest	0	0	9529

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 141891 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/2dc21e788874.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/2dc21e788874.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/2dc21e788874.tar.gz` (if generated)